

# Analysis of Prompt Engineering Techniques and Their Algorithmic Effect on Large Language Model Output

**1<sup>st</sup> Sarwagya Acharya**

iiDEPARTMENT<sub>dd</sub>

iiORGANIZATION<sub>dd</sub>

iiCITY<sub>dd</sub>, iiCOUNTRY<sub>dd</sub>

iiEMAIL SARWAGYA<sub>dd</sub>

**2<sup>nd</sup> Nitesh Pant**

iiDEPARTMENT<sub>dd</sub>

iiORGANIZATION<sub>dd</sub>

iiCITY<sub>dd</sub>, iiCOUNTRY<sub>dd</sub>

iiEMAIL NITESH<sub>dd</sub>

**3<sup>rd</sup> Brishav Joshi**

iiDEPARTMENT<sub>dd</sub>

iiORGANIZATION<sub>dd</sub>

iiCITY<sub>dd</sub>, iiCOUNTRY<sub>dd</sub>

iiEMAIL BRISHAV<sub>dd</sub>

July 26, 2026

## Abstract

Prompt engineering has rapidly evolved from ad-hoc text tweaks into a family of techniques that systematically reshape how large language models (LLMs) generate output. Yet the literature largely treats these techniques as linguistic patterns, leaving their algorithmic consequences under-quantified. This paper analyses prompt engineering techniques through the lens of the conditional distribution that an LLM defines over output tokens, and characterises how each technique shifts that distribution via prompting, decoding, and reasoning-trace construction. We taxonomise the major technique families, formalise their effect on sampling, attention, and output diversity, and report a controlled empirical comparison across representative tasks. The results show that the algorithmic effect of a prompt often dominates its lexical content, and we recommend evaluation protocols that make this effect measurable.

*Index Terms*—prompt engineering, large language models, chain-of-thought, decoding strategies, evaluation, algorithmic analysis

## 1 Introduction

Large language models (LLMs) are now routinely deployed for question answering, code synthesis, summarisation, and tool use, and the practical quality of an LLM application is shaped as much by the prompt that surrounds a model call as by the model weights themselves [1, 2]. A growing catalogue of prompt engineering techniques—zero- and few-shot exemplars, chain-of-thought (CoT) [3], self-consistency [4], tree-of-thoughts (ToT) [5], ReAct [6], and retrieval-augmented prompts [7]—reports substantial gains on established benchmarks, often with no change to the underlying model.

The dominant framing in this literature is empiri-

cal and qualitative: a technique is described by its prompting template and illustrated with selected outputs. While useful, this framing obscures a more fundamental question, namely *what does the technique change inside the model?* In this paper we argue that the principal techniques are best understood as algorithmic operations on the conditional distribution  $p_\theta(y \mid P, x)$  that an LLM defines over output tokens. Under this view, a prompt is not merely a textual wrapper; it is a control input that perturbs sampling, restructures attention, alters logit geometry, and conditions intermediate computational traces. The size of the resulting behavioural change often rivals, and sometimes exceeds, the difference between adjacent model scales.

The contributions of this paper are as follows.

- A taxonomy that organises prompt engineering techniques by the algorithmic object they modify: the prompt template, the decoding policy, or the reasoning trace.
- A formal framework that expresses each technique family as a transformation on  $p_\theta(y \mid P, x)$  and on the underlying arg max or sampling operator.
- A controlled empirical study that isolates the algorithmic effect of prompting from the lexical effect by holding the prompt template fixed and varying decoding parameters  $(T, k, p)$ , and vice versa.
- An evaluation protocol that we recommend for future prompt engineering research, designed to make algorithmic and lexical contributions separable.

The remainder of this paper is organised as follows. Section 2 reviews prior surveys and the foundational techniques that motivate this work. Section 3 introduces the taxonomy and notation. Section 4

formalises the algorithmic effect of each technique. Section 5 describes the experimental design. Section 6 reports the results, and Sections 7–9 discuss them, list limitations, and conclude.

## 2 Background and Related Work

The prompt engineering literature spans prompting patterns, decoding-time methods, and reasoning-trace construction. We organise the review along the same three axes used in our taxonomy (Section 3).

### 2.1 Prompting patterns as control inputs

Brown et al. [1] demonstrated that scaling model size produces in-context learning, in which a small set of exemplars in the prompt elicits task behaviour without weight updates. The result established prompting as a first-class interface to LLMs and motivated subsequent work on exemplar selection, ordering, and formatting. Min et al. [2] further showed that the ground-truth labels in exemplars contribute less than the input-output format, evidence that prompts act partly as structural scaffolds for the model’s own computations. Two findings sharpen this picture. Lu et al. [10] report that permuting a fixed exemplar set moves accuracy across a range wide enough to swamp the difference between competing methods, and Zhao et al. [11] trace part of that sensitivity to a prior bias in the label distribution that can be corrected without touching the wording of the prompt. Both results are difficult to explain if a prompt is regarded purely as a piece of natural language; both are unsurprising if it is regarded as a control input.

### 2.2 Reasoning-trace construction

Wei et al. [3] introduced chain-of-thought prompting, appending intermediate reasoning steps to exemplars and observing large gains on arithmetic, common-sense, and symbolic tasks. The technique reframes generation as a two-stage process: sample a reasoning chain  $r$ , then sample the answer  $y$  conditioned on  $r$ . Kojima et al. [9] showed that a single trigger phrase reproduces much of the effect without exemplars, which separates the contribution of the trace from that of the demonstrations. Wang et al. [4] extended CoT with self-consistency, sampling multiple chains and selecting the most frequent final answer; the gain over greedy CoT demonstrates that the underlying distribution is multimodal and that

marginalising over reasoning paths reallocates probability mass towards correct answers. Yao et al. [5] generalised chains to trees (tree-of-thoughts), Yao et al. [6] interleaved reasoning with tool calls in ReAct, and Madaan et al. [12] closed the loop by feeding the model’s own critique back into the context. Across these works, the common algorithmic move is to expose intermediate computation to the sampling operator.

### 2.3 Decoding-time and retrieval augmentation

Beyond reasoning traces, decoding-time methods such as temperature, top- $k$  [13], and nucleus (top- $p$ ) sampling [8] directly parametrise the output distribution; Meister et al. [14] argue that the useful criterion is closeness to the expected information content rather than raw probability mass. Retrieval-augmented generation (RAG) [7] couples the LLM to an external retriever, conditioning generation on documents fetched at inference time. While RAG is often presented as a knowledge-injection technique, it is, in our framing, a prompting technique: it changes the conditioning context  $P$  of  $p_\theta(y | P, x)$  and thereby changes the posterior over outputs.

### 2.4 Positioning of this paper

Existing surveys [15, 16] catalogue prompting techniques and, in the case of automated prompt search [17], optimise over them, but they do not formalise the algorithmic effect of the resulting prompts. Empirical comparisons exist for individual techniques, yet they typically vary the prompt while holding decoding fixed, which confounds the algorithmic and lexical contributions. To our knowledge, no prior work isolates the algorithmic effect of prompting through controlled ablations on both the prompt template and the decoding policy. This paper fills that gap by treating prompting and decoding as two parameters of the same sampling operator and measuring each contribution independently.

### 2.5 Common notation and primer

Throughout the paper we write  $P$  for a prompt template,  $x$  for the user input, and  $y$  for the model output. An LLM with parameters  $\theta$  defines a conditional distribution  $p_\theta(y | P, x)$ . Decoding is parameterised by a temperature  $T$ , a top- $k$  cutoff, and a nucleus threshold  $p$ . We treat CoT, self-consistency, and ToT as sampling procedures over reasoning traces  $r$  followed by answers  $y$ , and we treat retrieval augmen-

tation as a stochastic rewrite of  $P$ . These objects are formalised in Section 4.

### 3 Taxonomy of Prompt Engineering Techniques

A technique that improves a benchmark score tells us little by itself, because the improvement may originate at any of three points in the generation pipeline. We therefore classify techniques by the object they modify rather than by the surface form they take.

#### 3.1 Three operator classes

**Definition 1** (Prompting method). *A prompting method is a triple  $M = (\Phi, \Psi, \Omega)$ , where  $\Phi$  is a prompt-space operator acting on the conditioning context,  $\Psi$  is a decoding-space operator acting on the per-step token distribution, and  $\Omega$  is a trace-space operator acting on the set of intermediate sequences that are generated before the answer is emitted. Any component may be the identity.*

Definition 1 is deliberately coarse. Its purpose is to make the following claim checkable: two techniques that are described very differently in prose may occupy the same cell of the taxonomy, and two techniques presented as variants of one another may not. Table 1 assigns the principal techniques to cells and records the cost each incurs.

#### 3.2 Prompt-space operators

Prompt-space operators rewrite the conditioning context and leave both the decoding policy and the generated trace untouched. Instruction rewriting, role conditioning, exemplar selection and ordering, output-format scaffolding, and retrieval injection all belong here. What distinguishes them from one another is the amount of context they add and whether the added content is deterministic. Retrieval is the outlier: because the retriever returns a different document set for different inputs, the operator is stochastic even when the template around it is fixed, and any variance it introduces will be attributed to the prompt unless the retrieved set is logged.

#### 3.3 Decoding-space operators

Decoding-space operators leave the context untouched and reshape the per-step distribution before a token is drawn. Temperature scaling is a smooth reparametrisation; top- $k$  and nucleus sampling are support restrictions; beam search replaces per-step

sampling with an approximate search for a high-scoring sequence. These operators are the only ones in the taxonomy that can be applied without changing a single character of the prompt, which makes them the natural control condition for the experiments in Section 5.

#### 3.4 Trace-space operators

Trace-space operators change what the model generates before it commits to an answer. The simplest, zero-shot CoT, adds a trigger phrase and lets the model produce a chain; the most elaborate, tree-of-thoughts, maintains a frontier of partial traces and expands it under a value estimate. Self-consistency sits between the two: it does not alter the shape of an individual trace but draws several and aggregates them. We stress that trace-space operators are not free. Every one of them buys accuracy with tokens, forward passes, or both, and Table 1 records the exchange rate.

#### 3.5 Composition and interaction

The three operator classes do not compose independently, and the failure of independence is itself informative. Self-consistency is the clearest case: it requires  $T > 0$ , since  $m$  greedy samples are  $m$  copies of one chain, so the trace-space operator is only defined relative to a decoding-space operator that supplies diversity. Conversely, few-shot CoT couples  $\Phi$  and  $\Omega$  because the exemplars carry the traces. We therefore formulate the two hypotheses that the experiments in Section 5 are designed to test:

- H1 Holding the algorithmic structure of a prompt fixed, paraphrasing its wording produces a smaller change in task accuracy than moving between cells of Table 1.
- H2 The interaction between trace-space and decoding-space operators is significant, so reporting a trace-space technique without its decoding configuration underdetermines the result.

## 4 Algorithmic Framework

### 4.1 Base distribution

Let  $\mathcal{V}$  be the vocabulary and let  $y = (y_1, \dots, y_{|y|}) \in \mathcal{V}^*$  be an output sequence. An autoregressive LLM factorises

$z_t \in \mathbb{R}^{|\mathcal{V}|}$  by

$$p_\theta(y \mid P, x) = \prod_{t=1}^{|y|} p_\theta(y_t \mid y_{<t}, P, x),$$

where each factor is obtained from a logit vector

$$p_\theta(y_t = v \mid y_{<t}, P, x) = \frac{\exp(z_{t,v})}{\sum_{u \in \mathcal{V}} \exp(z_{t,u})}.$$

Table 1: Taxonomy of Prompt Engineering Techniques by Modified Algorithmic Object

| Technique                    | Operator        | Object modified          | Forward passes | Added tokens         | Stochastic? |
|------------------------------|-----------------|--------------------------|----------------|----------------------|-------------|
| Instruction rewriting        | $\Phi$          | conditioning context     | 1              | $O( I )$             | no          |
| Role / persona conditioning  | $\Phi$          | conditioning context     | 1              | $O(1)$               | no          |
| Few-shot exemplars           | $\Phi$          | conditioning context     | 1              | $O(n\bar{\ell})$     | no          |
| Format scaffolding           | $\Phi$          | conditioning context     | 1              | $O(1)$               | no          |
| Retrieval augmentation       | $\Phi$          | conditioning context     | 1+retrieval    | $O(n_d\bar{\ell}_d)$ | yes         |
| Temperature scaling          | $\Psi$          | per-step distribution    | 1              | 0                    | yes         |
| Top- $k$ truncation          | $\Psi$          | per-step support         | 1              | 0                    | yes         |
| Nucleus (top- $p$ ) sampling | $\Psi$          | per-step support         | 1              | 0                    | yes         |
| Beam search                  | $\Psi$          | sequence-level objective | $b$            | 0                    | no          |
| Constrained decoding         | $\Psi$          | per-step support         | 1              | 0                    | either      |
| Zero-shot CoT                | $\Omega$        | reasoning trace          | 1-2            | $O(1)$               | no          |
| Few-shot CoT                 | $\Phi + \Omega$ | context and trace        | 1              | $O(n\bar{\ell}_r)$   | no          |
| Least-to-most prompting      | $\Omega$        | trace decomposition      | $s$            | $O(s)$               | no          |
| Self-consistency             | $\Omega + \Psi$ | trace ensemble           | $m$            | 0                    | yes         |
| Tree-of-thoughts             | $\Omega + \Psi$ | trace search             | $O(bd)$        | $O(bd\bar{\ell}_r)$  | either      |
| ReAct                        | $\Phi + \Omega$ | trace and context        | $O(n_a)$       | $O(n_a\bar{\ell}_o)$ | yes         |
| Self-refine                  | $\Phi + \Omega$ | trace and context        | $2c$           | $O(c\bar{\ell}_r)$   | no          |